Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

# SDC Hackathon

# Technical Tools Declaration

**Hosted by Xiamen University Malaysia (XMUM) AI Club**

**Submission Deadline: 25 October 2026**

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

***Instagram:*** @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

**SDC HACKATHON 2026**

# Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

## 1. Team & Project Overview

| | |
|---|---|
| **Team Name** | Nabil The Goat |
| **Project / Solution Name** | sabave.ai |
| **Name (Leader)** | Nabil Hilmi Tsaqifi |
| **Email & Phone (Leader)** | nabiltsaqifi@gmail.com / +60 11 7244 6398 |
| **University Affiliation(s)** | Monash University Malaysia |
| **Live Prototype URL** | https://sabaveai-weather-marketing.replit.app |
| **Code Repository URL (public / access granted to judges)** | https://github.com/Joebilly112/sabaveai |

## 2. Architecture & Technology Stack

### 2.1 System Architecture Overview

*Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.*

***[Maximum 200 words]***

User (browser, as Staff or Owner) ⟶ Next.js 14 web app hosted on Replit (React + Tailwind UI) ⟶ Next.js API routes (server, Node.js 22). The server (a) calls the Open-Meteo Air Quality and Weather Forecast APIs for PM2.5, rainfall and thunderstorm forecasts at each business's location; (b) runs the trigger engine and deterministic campaign rules; (c) calls the DeepSeek API (OpenAI-compatible) to write bilingual captions and summarise brand style, with a template fallback; (d) enforces roles, budget caps and never-say checks in the approval service; and (e) publishes through a swappable social connector (a mock Meta connector in the prototype). Business Brand Kits, drafts and the append-only activity log are held in an in-memory store seeded from synthetic JSON files (Firestore is planned for production). API keys are stored in Replit Secrets and used only server-side. Source code is hosted on GitHub.

### 2.2 Languages, Frameworks & Libraries

*List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.*

| Name & Version | Purpose in Project | License |
|---|---|---|
| Next.js 14 | Web framework (UI and API routes) | MIT |
| React 18 | User interface | MIT |
| TypeScript 4.9 | Programming language | Apache 2.0 |
| Tailwind CSS 3 | Styling | MIT |
| Node.js 22 | Server runtime | MIT |

## 3. APIs & Cloud Services

### 3.1 External APIs

*List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.*

| API & Provider | Purpose in Project | Access Tier (free / paid / sponsor-provided) |
|---|---|---|
| Open-Meteo Air Quality API (Open-Meteo) | PM2.5 forecast that triggers haze campaigns | Free |
| Open-Meteo Weather Forecast API (Open-Meteo) | Rain and thunderstorm forecast trigger | Free |
| DeepSeek Chat Completions API (DeepSeek) | Writes bilingual campaign copy and summarises brand style | Paid (pay-as-you-go) |
| Meta Graph API (Meta) | Simulated only; no real calls are made in the prototype | N/A (mocked) |

### 3.2 Cloud Platforms & Deployment

*List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.*

| Service & Provider | Purpose in Project | Plan / Region |
|---|---|---|
| Replit | Development environment, secrets storage (Replit Secrets) and live deployment of the prototype | Replit Core (sponsor-provided) / default region |
| GitHub | Public source code repository and version history | Free / N/A |

*Deployment: the prototype is deployed as a Next.js app on Replit and is reachable at the Live Prototype URL in Section 1.*

# 4. AI Tools, Models & Datasets

## 4.1 AI Models Used in the Solution

*List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.*

| Model & Version | Source / Provider | Usage (pre-trained / fine-tuned / self-trained) | License / Terms |
|---|---|---|---|
| DeepSeek latest Pro model deepseek-v4-pro | DeepSeek | Pre-trained, called via API | DeepSeek API Terms of Service |

## 4.2 Datasets

*List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.*

| Dataset Name | Source / URL | Type (public / private / self-collected / synthetic) | License | Personal Data? (Y/N) |
|---|---|---|---|---|
| SME business profiles and Brand Kits | Self-created (/data in repo) | Synthetic | Apache 2.0 | N |
| Offline fallback forecast | Self-created (/data in repo) | Synthetic | Apache 2.0 | N |
| Air quality and weather forecasts | https://open-meteo.com | Public | CC BY 4.0 | N |

### 4.3 AI-Assisted Development Tools

*Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.*

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☒ Yes ☐ No

If yes, list them below:

| Tool & Version | Used For | Extent of Use (minor / moderate / major) |
|---|---|---|
| Claude (Anthropic), Claude Opus 5.5 via claude.ai | Interpreting the brief, feedback on our idea, structuring requirements, writing the Replit build prompt, drafting the submission documents | Major |
| Replit Agent (Replit) | Generating application code from our written prompt | Major |

**Final build prompt given to Replit Agent:** Prompts given to Replit Agent

Declared as required by the SDC Hackathon 2026 Technical & Tools Declaration (section 4.3). The product idea, problem and feature decisions are the team's own; the prompts below were drafted with help from Claude (Anthropic) and then given to Replit Agent to generate code.

**1. Main build prompt (final version, DeepSeek)**

Build a web app called "sabave.ai" (save + brave): a weather-triggered marketing automation module for "Orbyte", a shared business hub platform for Malaysian micro and small businesses (SMEs). When a weather signal (haze, or heavy rain/thunderstorm) is forecast for a business's location, sabave.ai automatically prepares 2-3 marketing campaign options in that business's own brand style. Staff edit a draft, the Owner approves it within a budget cap, the campaign is "published" to Facebook/Instagram through a MOCKED Meta connector, and every step is recorded in an append-only activity log.

The app must demonstrate one complete 5-step journey:

1. TRIGGER: a weather signal crosses its threshold (or a "Simulate" button is pressed)

2. CONNECTED DATA: the business's Brand Kit and the forecast are loaded

3. DECISION LOGIC: deterministic rules choose the campaign options and budgets; the LLM only summarises brand style and writes copy

4. HUMAN APPROVAL: only the Owner role can approve and publish (enforced on the server)

5. VISIBLE OUTCOME: a simulated publish result is shown and logged

==========

STACK AND CONSTRAINTS

==========

- Next.js 14 (App Router), TypeScript (strict), React 18, Tailwind CSS, Node 22. No UI kits beyond Tailwind.

- Must run and deploy on Replit. Dev: `next dev -H 0.0.0.0 -p 3000`. Production: `next build && next start -H 0.0.0.0 -p 3000`. Configure .replit for both.

- NO database. JSON seed files in /data plus an in-memory server store implemented as a singleton on globalThis (so it survives hot reloads). Uploaded images are stored in memory as base64 data URLs (max 2 MB each, PNG/JPEG/SVG/WebP only).

- External weather APIs (both free, no key). Use a 5-second timeout on each; on failure, use the bundled offline forecast and label it "offline data":

  * Air quality: https://air-quality-api.open-meteo.com/v1/air-quality?latitude={lat}&longitude={lon}&hourly=pm2_5&forecast_days=5&timezone=Asia%2FKuala_Lumpur

  * Weather: https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&daily=precipitation_sum,precipitation_probability_max,weather_code&forecast_days=5&timezone=Asia%2FKuala_Lumpur

- LLM: DeepSeek, via its OpenAI-compatible Chat Completions API, server-side only.

  * POST {DEEPSEEK_BASE_URL}/chat/completions with the header "Authorization: Bearer {DEEPSEEK_API_KEY}".

  * Env vars from Replit Secrets: DEEPSEEK_API_KEY (required for AI features), DEEPSEEK_BASE_URL (default "https://api.deepseek.com"), DEEPSEEK_MODEL (optional override). Never send the key to the browser, never prefix it with NEXT_PUBLIC_, never log it, never hard-code it.

  * Model selection: use the latest DeepSeek Pro model. If DEEPSEEK_MODEL is set, use it. Otherwise call GET {DEEPSEEK_BASE_URL}/models once on first use (cache the result) and pick the newest model whose id contains "pro"; if none, fall back to "deepseek-chat". Log the chosen model id (not the key) to the server console.

  * Request JSON output with response_format { "type": "json_object" }, and include the word "JSON" plus an example schema in the system prompt. If the response contains reasoning content, ignore it and parse only the final message content. Validate the parsed JSON; on invalid JSON retry once, then fall back to a template.

  * Timeout of 30 seconds per call (Pro models can be slow); show a spinner in the UI.

  * DeepSeek is TEXT ONLY: never send images to it.

  * Every LLM call has a template/manual fallback so the journey never breaks.

  * The LLM NEVER decides budgets, which option is chosen, or whether to publish.

- Meta is MOCKED. No real Meta/Facebook/Instagram/WhatsApp/TikTok API calls, no real payments or ad spend. Label all publish results "simulated".

- Synthetic data only. Business-level data only, no customer personal data (Malaysian PDPA).

- Root files: LICENSE (full Apache 2.0 text), .env.example (DEEPSEEK_API_KEY=, DEEPSEEK_BASE_URL=, DEEPSEEK_MODEL= with empty values), .gitignore (node_modules, .next, .env*, but keep .env.example), README.md. Never commit secrets.

- Login and roles are simulated; no auth library.

==========

FOLDER STRUCTURE

==========

/app

  layout.tsx

  page.tsx  (dashboard)

  onboarding/page.tsx  (Brand Kit form, new or edit)

  log/page.tsx  (activity log)

  /api/businesses/route.ts  GET list, POST create

  /api/businesses/[id]/route.ts  GET one, PATCH brand kit

  /api/forecast/route.ts

/api/trigger/route.ts

/api/generate/route.ts

/api/drafts/route.ts

/api/approve/route.ts

/api/brand/analyze-poster/route.ts

/api/brand/analyze-website/route.ts

/api/log/route.ts

/api/stats/route.ts

/components

TopBar, StatsStrip, SignalForecastCard, AlertCard, OptionCard, DraftEditor,

PosterPreview, ApprovalPanel, PublishResult, LogTable, BrandKitForm,

ImageUploader, ToneChips, ProductListEditor

/lib

types.ts, store.ts, users.ts, activityLog.ts

/signals: signal.ts (interface), hazeSignal.ts, rainSignal.ts, registry.ts

triggerEngine.ts, campaignRules.ts, copyGenerator.ts, neverSay.ts,

brandAnalyzer.ts, llm.ts (DeepSeek client), approvalService.ts, metaConnector.ts,

colorExtract.ts (client-side)

/data

businesses.json, offlineForecast.json

/public/seed

SVG placeholder logos and past posters for the 3 seed businesses

/docs/.gitkeep

==========

TYPES (lib/types.ts)

==========

- Role = "staff" | "owner"; User { id, name, role }

- Product { name, priceRM }

- BrandKit {

  logoDataUrl?, logoPath?, brandColours: string[] (1-3 hex values),

  pastPosterDataUrl?, pastPosterPath?, posterColours?: string[], posterDescription?, posterStyleSummary?,

  websiteUrl?, websiteVoiceSummary?,

  ethos: string, toneChips: string[],

  products: Product[], targetCustomers: string,

  delivery: boolean,

  channels: ("facebook"|"instagram"|"whatsapp"|"tiktok")[],

  language: "bm"|"en"|"both",

  neverSay: string[],

  monthlyAdBudgetCapRM: number, approverUserId: string,

  baselineWeeklySalesRM: number, lastBadWeatherDipPct: number

 }

- Business { id, name, type: "cafe"|"laundromat"|"retail"|"other", city, lat, lon, socialPages: { facebookPageId, instagramHandle }, brandKit: BrandKit }

- SignalId = "haze" | "rain"

- SignalDay { date, value, unit, level: 0|1|2|3, levelLabel, detail? }

- SignalForecast { signalId, businessId, days: SignalDay[], source: "open-meteo"|"simulated"|"offline", fetchedAt }

- WeatherAlert { id, businessId, signalId, affectedDates, peakDate, peakLevelLabel, daysAhead, createdAt, open: boolean }

- CampaignOption { id, signalId, kind, title, description, reason, offer, suggestedBudgetRM }

- CampaignCopy { headline, caption_bm, caption_en, hashtags: string[], cta, rationale, source: "ai"|"template", model? }

- DraftStatus = "draft" | "pending_approval" | "published" | "rejected"

- CampaignDraft { id, businessId, alertId, option, copy, budgetRM, channels, status, createdBy, approvedBy?, rejectReason?, publishResult?, history: {at, by, action}[] }

- PublishResult { platforms: string[], postIds: Record<string,string>, adId?, status: "published (simulated)", publishedAt }

- LogEntry { id, timestamp, actor, role, businessId, action, details }

==========

MODULES AND RESPONSIBILITIES

==========

users.ts: two simulated users, { id: "u1", name: "Aisyah", role: "staff" } and { id: "u2", name: "Mr. Tan", role: "owner" }.

store.ts: globalThis singleton holding businesses (loaded from JSON on first access), forecasts, alerts, drafts and logs, with simple get/set helpers.

activityLog.ts: append(entry) and list(businessId?). Append-only: no update or delete. Actions: FORECAST_CHECKED, SIGNAL_SIMULATED, ALERT_TRIGGERED, OPTION_SELECTED, DRAFT_GENERATED, DRAFT_EDITED, NEVER_SAY_BLOCKED, SUBMITTED_FOR_APPROVAL, APPROVED, REJECTED, PUBLISHED, BUDGET_BLOCKED, ACTION_DENIED, BRAND_KIT_CREATED, BRAND_KIT_UPDATED, POSTER_ANALYZED, WEBSITE_ANALYZED, ANALYSIS_FALLBACK, AI_FALLBACK.

signals/signal.ts: a pluggable interface so new signals (e.g. heatwave) can be added just by registering a new file:

interface WeatherSignal { id: SignalId; label: string; icon: string; fetchForecast(b: Business): Promise<SignalForecast>; simulate(b: Business): SignalForecast; triggerLevel: number }

signals/registry.ts: exports all registered signals.

signals/hazeSignal.ts: Open-Meteo pm2_5, grouped into daily max values (µg/m³). Levels: 0 Good (<35), 1 Moderate (35-55), 2 Unhealthy (55-150), 3 Very Unhealthy (>150). triggerLevel = 2. simulate() returns the daily values [45, 90, 165, 120, 50].

signals/rainSignal.ts: Open-Meteo daily precipitation_sum (mm) and weather_code. Levels: 0 Light (<10 mm), 1 Moderate (10-30), 2 Heavy (30-60, or any thunderstorm code 95-99), 3 Very Heavy (>60). detail = "Thunderstorm" when the code is 95-99. triggerLevel = 2. simulate() returns [5, 40 + thunderstorm, 75 + thunderstorm, 35, 8].

triggerEngine.ts: evaluate(forecast, business): if any day's level >= triggerLevel, create a WeatherAlert (affected dates, peak, daysAhead = days from today to the first triggering day). Don't create a duplicate open alert for the same business + signal + dates. Log ALERT_TRIGGERED.

campaignRules.ts (deterministic and explainable, NO AI): an option catalogue per signal, filtered by business type and Brand Kit:

HAZE: "delivery_push" (only if delivery = true), "stay_home_bundle" (cafe: family meal/drinks pack; laundromat: pickup + wash-and-fold; retail: online order + click-and-collect), "clean_air_walk_in" ("We're open, comfortable indoor air" with a small in-store offer).

RAIN: "rainy_day_delivery" (only if delivery = true), "shelter_comfort" ("Come shelter with us" comfort offer, e.g. hot drinks, express dry), "rain_check_voucher" (a voucher redeemable on the next dry day).

- Return 2-3 options. offer uses the business's real products and prices (e.g. "RM2 off Nasi Lemak Ayam (RM8 -> RM6)").

- suggestedBudgetRM = cap × (0.3 for level 2, 0.5 for level 3) × min(1, triggeringDays / 3), rounded to the nearest RM10, minimum RM20, never above the cap.

- reason is a plain sentence built from data, e.g. "Very Unhealthy haze is forecast in Shah Alam on 28-30 Oct. Last time bad weather hit, your sales dropped 22%. You deliver, so a delivery push keeps orders coming while walk-ins fall."

neverSay.ts: findViolations(text, neverSay[]): case-insensitive whole-word/phrase match across headline, captions, hashtags and CTA. Always also blocks health/medical claims (e.g. "cure", "protects your lungs", "prevents illness", "kills germs").

llm.ts (DeepSeek client): generateJSON({ systemPrompt, userPrompt, exampleSchema }) -> parsed object or throws. Handles model selection and caching, response_format json_object, the 30-second timeout, one

retry on invalid JSON, and ignoring reasoning content. If DEEPSEEK_API_KEY is missing, it throws immediately so callers fall back. Also export getActiveModel() for UI badges.

copyGenerator.ts: generateCopy(business, alert, option): builds the prompt from business-level fields only (name, type, city, products, target customers, ethos, tone chips, posterStyleSummary, websiteVoiceSummary, language, neverSay) plus the signal, level, dates and option. The prompt asks for JSON { headline (≤ 8 words), caption_bm, caption_en (each ≤ 60 words, natural Malaysian tone, matches the brand voice), hashtags (3-6), cta, rationale (1 sentence) }, with no health/medical claims and avoiding every neverSay term. If the result contains violations, retry once with the violations listed; if it still fails, use the template copy and log NEVER_SAY_BLOCKED. On any error, use the template copy and log AI_FALLBACK. Always return both captions; the UI highlights the preferred language. templateCopy() builds a decent caption from the option, offer, business name and tone. source = "ai" (with the model id) or "template".

brandAnalyzer.ts:

- analyzePoster({ colours, description, toneChips }): colours come from client-side extraction of the uploaded past poster; description is the owner's short description of it. DeepSeek returns JSON { layout, typographyStyle, tone, summary }. Fallback (no AI): a summary built from the colours, chips and description; log ANALYSIS_FALLBACK.

- analyzeWebsite(url): server fetch with a 6-second timeout, a browser-like User-Agent, http(s) only, max 1 MB, blocking localhost/private IP ranges; extract the title, meta description and visible text (strip script/style tags, first 4000 characters); DeepSeek returns JSON { voiceSummary, keyThemes: string[], suggestedTone: string[] }. Fallback: return { fallback: true } and the UI uses the ethos text.

colorExtract.ts (client-side): draw an uploaded image on a canvas, sample pixels, ignore near-white, near-black and transparent pixels, bucket the colours, and return the top 3-5 as hex values. Used for both the logo (brand colours, editable with colour inputs) and the past poster (posterColours).

approvalService.ts (enforced on the SERVER):

- Staff: create and edit drafts (copy, offer, budget, channels) and submit (draft -> pending_approval).

- Owner: everything staff can do, plus approve (pending_approval -> published, which calls metaConnector) or reject (-> rejected, reason required).

- Budget must be within RM0 and the cap; if it's over, return a 400 error and log BUDGET_BLOCKED.

- Submit and approve both run neverSay on the current copy; if there's a violation, return a 400 error listing the terms and log NEVER_SAY_BLOCKED.

- A wrong role returns 403 and logs ACTION_DENIED.

- Every transition is appended to draft.history and the activity log.

metaConnector.ts:

interface SocialConnector { publishPost(draft, business): Promise<PublishResult>; createAd(draft, business, budgetRM): Promise<{ adId: string }> }

class MockMetaConnector implements SocialConnector: waits ~800ms; returns fake IDs like "fb_post_" + 6 random digits, "ig_post_" + 6 digits and "ad_" + 6 digits, with status "published (simulated)". Only Facebook/Instagram are "published"; WhatsApp/TikTok are shown as "coming soon".

getConnector() factory, with a comment explaining it is designed as a reusable Orbyte connector that any Orbyte module can call, and that a real Meta Graph API connector can be swapped in later.

==========

API ROUTES

==========

The client sends the simulated user in the header "x-user-id" (u1 or u2). The server resolves the role from users.ts and never trusts a role sent by the client.

- GET /api/businesses -> Business[] (summary) | POST /api/businesses (BrandKit form) -> Business

- GET /api/businesses/[id] -> Business | PATCH /api/businesses/[id] -> updated Business

- GET /api/forecast?businessId=&signalId= -> SignalForecast

- POST /api/trigger { businessId, signalId, simulate?: boolean } -> { forecast, alert | null, options: CampaignOption[] }

- POST /api/generate { businessId, alertId, optionId, useAI?: boolean } -> CampaignDraft (status "draft", budget = suggested)

- PATCH /api/drafts { draftId, copy?, offer?, budgetRM?, channels?, action?: "submit" } -> CampaignDraft

- POST /api/approve { draftId, decision: "approve"|"reject", budgetRM?, reason? } -> CampaignDraft (owner only)

- POST /api/brand/analyze-poster { colours, description, toneChips } -> analysis | { fallback: true, summary }

- POST /api/brand/analyze-website { url } -> analysis | { fallback: true }

- GET /api/log?businessId= -> LogEntry[] newest first

- GET  /api/stats?businessId= -> { campaignsLaunched, avgDaysAhead, minutesSaved } where minutesSaved = campaignsLaunched × 85 (assumes ~90 min manually vs ~5 min with sabave.ai)

==========

SEED DATA (/data/businesses.json), synthetic, with COMPLETE Brand Kits

==========

Create SVG placeholder logos (initials in brand colours) and simple SVG "past posters" in /public/seed, and pre-fill posterColours, posterDescription, posterStyleSummary and websiteVoiceSummary so the demo needs no live analysis. Use fake example websites (e.g. https://example.com/selerapagi).

1. "Kopitiam Selera Pagi": cafe, Shah Alam (3.0733, 101.5185). Colours #B45309, #FDE68A. Ethos: "Old-school kopitiam warmth, affordable breakfast for working families." Tones: traditional, family, halal-friendly. Products: Kopi O RM2.50, Nasi Lemak Ayam RM8, Roti Bakar Kaya RM4. Target: office workers and families nearby. delivery true. Channels: facebook, instagram, whatsapp. Language: both. Never say: ["pork", "alcohol", "cheapest in town"]. Cap RM600, baseline RM9000/week, last bad-weather dip 22%.

2. "Dobi Ceria Express": laundromat, Petaling Jaya (3.1073, 101.6067). Colours #0369A1, #BAE6FD. Ethos: "Fast, clean and friendly self-service laundry for busy people." Tones: modern, playful. Products: Wash RM5, Dry RM5, Wash & Fold RM15/5kg, Duvet Clean RM25. Target: students and young working adults. delivery false (pickup service only). Channels: facebook, instagram, tiktok. Language: both. Never say: ["kills all germs", "100% hygienic"]. Cap RM300, baseline RM4500, dip 15%.

3. "Butik Anggun": clothing retail, Johor Bahru (1.4927, 103.7414). Colours #BE185D, #FCE7F3. Ethos: "Elegant modest fashion at honest prices." Tones: premium, traditional. Products: Baju Kurung RM129, Tudung Bawal RM35, Batik Shirt RM89. Target: women aged 25-50 shopping for work and festive occasions. delivery true. Channels: facebook, instagram, whatsapp. Language: bm. Never say: ["cheap", "murah gila", "Butik Seri"]. Cap RM800, baseline RM12000, dip 28%.

Give each a fake facebookPageId and instagramHandle; approver = u2.

/data/offlineForecast.json: a mostly calm 5-day forecast for both signals.

==========

UI AND FLOW (simple, clean, mobile-first, Tailwind)

==========

- TopBar: "sabave.ai · an Orbyte module" wordmark, business selector, user switcher ("Aisyah · Staff" / "Mr. Tan · Owner"), and links to Dashboard, Brand Kit and Activity Log. Selections are kept in React state (localStorage optional, wrapped in try/catch).

- StatsStrip: campaigns launched, average days ahead of the weather event, estimated minutes saved.

- Dashboard: one SignalForecastCard per registered signal (Haze, Rain). Each has 5 day tiles with the date, value + unit, and a colour-coded level (green/yellow/orange/red; a thunder icon for thunderstorms), a source badge, and the buttons "Check forecast" and "Simulate".

- AlertCard: "⚠ {levelLabel} {signal} forecast for {city}, {dates}. Last time bad weather hit, your sales dipped {x}%. You're {n} days ahead."

- OptionCards: title, description, offer, a "Why this?" reason, suggested budget, and a "Prepare campaign" button (with a toggle "Write copy with AI", on by default if a key is configured).

- DraftEditor: editable headline, BM caption, EN caption, hashtags, offer, budget (cap shown), channel checkboxes, a copy source badge ("AI (DeepSeek · {model})" or "template"), a "Regenerate with AI" button, and inline never-say warnings as you type. PosterPreview sits beside it on desktop and stacks on mobile.

- PosterPreview: a square HTML/CSS poster using the logo, brand colours, business name, headline, offer and CTA, with a subtle signal-themed background (hazy gradient for haze, rain streaks for rain). The style adapts to the tone chips (e.g. premium = serif headline and more whitespace; playful = rounded and bold) and to posterStyleSummary where possible.

- ApprovalPanel:

  * Staff: "Submit for approval", then "Waiting for owner approval".

  * Owner, when pending: "Approve & publish" and "Reject" (reason required), with a budget field and an inline error if it's over the cap.

  * Approve buttons are hidden for staff, and the server rejects them anyway.

- PublishResult: a green panel with "Published to Facebook & Instagram (simulated)", post IDs, ad ID, budget and time.

- /onboarding (Brand Kit): BrandKitForm with name, type, city + lat/lon (preset dropdown of major Malaysian cities that fills lat/lon automatically, plus manual entry), logo upload (preview + extracted brand colours, editable), past poster upload (preview + extracted colours) with a short description field and an "Analyze style" button (shows the DeepSeek summary or the no-AI summary), website URL with an "Analyze website" button (shows the voice summary or "Couldn't read this site; we'll use your ethos instead"), ethos textarea, tone chips, products editor, target customers, delivery toggle, channels, language, never-say list (chips),

budget cap, approver, baseline sales and last dip %. Save -> POST or PATCH, then log BRAND_KIT_CREATED or BRAND_KIT_UPDATED. Also allow editing a seeded business.

- /log: LogTable with time, actor, role, signal, action and details, filterable by business, newest first.

- Friendly empty states, loading spinners and error toasts everywhere.

Demo path that must work end to end: select Kopitiam -> Haze card -> Simulate -> alert and options appear -> Prepare campaign -> as Aisyah, edit the caption and submit -> switch to Mr. Tan -> approve within budget -> simulated publish shown -> the Activity Log shows every step.

==========

BUILD IN STAGES. STOP after each stage, summarise what was built and how to test it, then WAIT for me to say "continue".

==========

STAGE 1: Project setup (Next.js, Tailwind, .replit, LICENSE, .env.example, .gitignore), types, store, users, activityLog, seed data with full Brand Kits + SVG placeholders, the signal interface + registry + hazeSignal (real, simulate, offline), /api/businesses (GET), /api/forecast, TopBar, and the Haze SignalForecastCard.

STAGE 2: triggerEngine, campaignRules (haze options), /api/trigger, AlertCard and OptionCards with reasons.

STAGE 3: Drafts with TEMPLATE copy only, neverSay, approvalService with server-side role/budget/never-say checks, MockMetaConnector, /api/generate, /api/drafts, /api/approve, /api/log, DraftEditor, a basic PosterPreview (logo + colours + text), ApprovalPanel, PublishResult and the /log page. The full 5-step haze demo path must work at the end of this stage.

STAGE 4: llm.ts (DeepSeek client with model auto-selection) + copyGenerator with JSON validation, a never-say retry and template fallback, the AI/template badge, "Regenerate with AI", and the tone-aware PosterPreview.

STAGE 5: The Brand Kit onboarding page: BrandKitForm, logo and poster upload + colorExtract, /api/brand/analyze-poster (text-only DeepSeek), website URL + /api/brand/analyze-website, all fallbacks, POST/PATCH businesses.

STAGE 6: rainSignal + rain campaign rules + the Rain SignalForecastCard, StatsStrip + /api/stats, UI polish and mobile check, and a README covering: what sabave.ai does, the 5-step journey, how to run on Replit, the required Secrets (DEEPSEEK_API_KEY etc.), that DeepSeek is used via its OpenAI-compatible API only for copywriting and brand summaries, that Meta and roles are simulated, synthetic data and PDPA notes (business-level data only, uploads kept in memory, secrets server-side), how to add a new weather signal, and the roadmap (real Meta Graph API connector, WhatsApp Business/TikTok, Firestore persistence, scheduled automatic forecast checks, heatwave signal, image-based poster analysis, campaign performance tracking).

Keep the code readable, with short comments explaining each module's role in the journey.

## 2. UI touch-up prompt

Make a few small UI and workflow touch-ups. Do NOT change the data model, API routes, role/approval logic, or anything that already works. Keep the existing visual style (colours, fonts, cards). Do all of these in one pass, then stop and summarise.

1. JOURNEY PROGRESS BAR: At the top of the dashboard (under the header), add a slim 5-step progress indicator: "1 Forecast → 2 Alert → 3 Options → 4 Approval → 5 Published". Highlight the current step based on state: no alert = step 1; alert exists = step 2-3; a draft is pending approval = step 4; a draft is published = step 5. Completed steps get a check icon.

2. AUTO-SCROLL: After clicking "Prepare campaign", smoothly scroll to the draft editor and open the new draft automatically. After "Simulate", scroll to the alert card.

3. HIDE EMPTY DRAFTS: Hide the whole "Review, approve, publish" section until at least one draft exists, instead of showing the empty "No drafts yet" / "Open a draft" boxes.

4. ROLE HANDOFF: When a Staff user submits a draft, show a clear banner in the draft panel: "Sent to Mr. Tan (Owner) for approval" with a button "Switch to Owner view" that changes the user switcher to Mr. Tan. When the Owner views a pending draft, show a highlighted "Awaiting your approval" badge.

5. DIFFERENT BUDGETS PER OPTION: Right now all options suggest the same budget. In campaignRules, multiply the suggested budget by option type: delivery push ×1.0, bundle ×0.8, in-store/walk-in ×0.6 (still rounded to nearest RM10, min RM20, never above the cap).

6. SHORTER "WHY THIS?": The alert card already shows the dates and the past sales dip, so in each option's reason remove the repeated sentence about the forecast dates and the 22% dip. Keep only the option-specific reason (one sentence).

7. CONSISTENT SEVERITY LABELS: The forecast legend says "Low / Elevated / High / Very high" but the tiles say "Good / Moderate / Unhealthy / Very Unhealthy". Make the legend use the same labels as the tiles. Also tint each day tile's background by severity (very light green / yellow / orange / red) so bad days stand out at a glance.

8. SMALL POLISH: Widen the business selector so "Kopitiam Selera Pagi" isn't cut off. Change the header tagline from "A WEATHER SIGNAL BY ORBYTE" to "AN ORBYTE MODULE". Change the dashboard greeting "Good to see you." to "Weather ahead for {business name}".

After making the changes, run the app and confirm this path still works end to end: select Kopitiam → Simulate → Prepare campaign → as Aisyah submit → Switch to Owner view → Approve & publish → Activity Log shows every step.

Note: Replit Agent chose the final implementation stack itself (a pnpm monorepo with a React + Vite frontend and an Express API server) rather than the Next.js stack named in the original prompt.

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

***[Maximum 100 words]***

Product decisions, the problem framing and the idea came from our own conversations with SME owners. AI tools helped interpret the brief, structure requirements and generate code from our written prompt. We built in small stages, manually tested each stage in Replit before committing, and ran the full demo journey end to end. We reviewed the generated code for hard-coded secrets and kept all keys in Replit Secrets. Inside the app, AI only drafts copy: deterministic rules and a human owner make every decision. We take full responsibility for the submitted work.

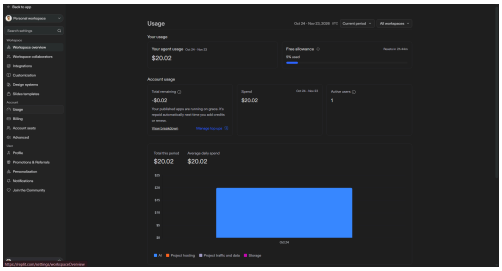
## 5. Sponsor-Provided Resources

*Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.*

Did your project use any official SDC sponsor-provided resource?

☒ Yes ☐ No

If yes, list them below:

| **Sponsor** | **Resource (API / credits / platform / data)** | **How It Was Used in the Project** |
|---|---|---|
| Replit | Platform (Replit Core) | Development, secrets storage and live deployment of the prototype  |
| RunPod | Credits | Not used (no GPU workload) |

**Replit proof of use:** https://replit.com/@nabiltsaqifi/sabaveai-weather-marketing

https://sabaveai-weather-marketing.replit.app

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

***[Maximum 100 words]***

Replit is where we developed, stored secrets for and deployed the live prototype, so it was central to our workflow. It is not essential to the core functionality: sabave.ai is a standard Next.js application and could be deployed on another Node.js host (e.g. Vercel or Firebase Hosting) without code changes. The RunPod credit was not used because the project has no GPU workload.

## 6. Security, Privacy & Responsible Use

### 6.1 Data Handling & Privacy

*What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.*

***[Maximum 150 words]***

No end-customer personal data is collected. The app stores business-level data only (business name, location, products, brand assets and budget cap) plus simulated user names for the Staff and Owner roles. In the prototype, all data is synthetic and kept in server memory; it is cleared when the server restarts, and uploaded images are kept in memory only. Data sent to third parties: business location coordinates to Open-Meteo, and business-level brand and campaign details (no personal data) to DeepSeek for copywriting. API keys are stored in Replit Secrets, used only server-side, and never sent to the browser or committed to GitHub. In production we would follow Malaysia's PDPA 2010: consent at onboarding, purpose limitation, a defined retention period, encrypted storage and role-based access.

### 6.2 Known Limitations & Risks

*Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.*

***[Maximum 150 words]***

**Hallucination or off-brand copy:** LLM output is constrained to strict JSON, checked against each business's never-say list and a health-claim filter, and always reviewed and approved by the owner. **Provider dependency and rate limits:** each DeepSeek call has a timeout and a deterministic template fallback, and the client is OpenAI-compatible so the provider can be swapped. **Forecast accuracy:** Open-Meteo forecasts are model-based and can be wrong; thresholds are conservative, the owner makes the final decision, and an offline fallback keeps the app usable. **Text-only model:** poster style is inferred from extracted colours and the owner's description rather than image analysis. **Simulated publishing:** real Meta access needs app review; the fallback is a downloadable poster and caption. **Storage:** the in-memory store resets on restart.

## 7. Compliance & Readiness Checklist

*Tick all boxes that apply before submitting.*

- ☒ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☒ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☒ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☒ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☒ Any use of official SDC sponsor resources has been declared in Section 5.
- ☒ The prototype described here is accessible to judges through the links provided in Section 1.
- ☒ All fields in this template have been completed within the word limits stated.

## 8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



25 October 2026